PRODUCT & ENGINEERING SPECIFICATION

WeddingOS

API Design Document

Version 1.0 | Design baseline | 09 November 2026

Document field	Value
Product	WeddingOS - collaborative Indian wedding management platform
Document	API Design v1.0
Status	Baseline for implementation; listed assumptions should be validated during build
Backend	Node.js + Express.js (JavaScript ES modules)
Database	MongoDB Atlas + Mongoose
API style	REST over HTTPS, JSON payloads, versioned under /api/v1
Related specifications	PRD v1.0, System Design Document v1.0, Database Design v1.0

Purpose

Define the HTTP API contracts, access rules, request and response formats, validation, key workflows, and security controls required to implement WeddingOS consistently across the React frontend and Express backend.

Design principle

The API is organized around user workflows and wedding-level access boundaries, not as a one-to-one mirror of MongoDB collections.

Document control

Version	Date	Change
1.0	09 Nov 2026	Initial consolidated API specification based on the approved PRD, SDD, database model, and decisions made during API design.

Contents

- 1. Scope and API design principles
- 2. API conventions
- 3. Authentication and authorization model
- 4. Role and lifecycle permission matrix
- 5. Endpoint catalogue
- 6. Detailed endpoint contracts
- 6.1 Authentication and account security
- 6.2 Weddings and lifecycle
- 6.3 Membership and invitations
- 6.4 Events
- 6.5 Categories and guest families
- 6.6 Guest-facing invitation and RSVP APIs
- 6.7 Expenses and payments
- 6.8 Vendors and vendor directory
- 6.9 Tasks
- 6.10 Galleries and media uploads
- 6.11 Dashboard and public wedding page
- 7. Standard response and error catalogue
- 8. Cross-module workflows and transactions
- 9. Security, validation and operational controls
- 10. Database alignment and required model updates
- 11. Implementation sequence
- 12. Open assumptions and deferred details

1. Scope and API design principles

This document specifies the v1.0 REST API for the WeddingOS MVP. It covers account authentication, wedding workspaces, membership, invitations, events, guest families, RSVPs, expenses, payments, vendors, tasks, galleries, dashboard summaries, and public/guest access.

1.1 In scope

- Email/password registration and login, Google authentication, email verification, password reset, refresh-token rotation, and device sessions.
- One created/owned wedding per user, while allowing participation in multiple weddings through active memberships.
- Two equal Admins per wedding; Managers with broad operational access but no Admin or lifecycle control.
- Seven-day, single-use invitations for Admin and Manager roles; recipient must authenticate and match the invited verified email.
- Guest-family records, event-specific invitations, family-level RSVP, CSV import preview/commit, and token-based guest pages.

- Record-only expenses and instalments; manually saved vendor information; event-based task management.
- Event galleries with separate view/upload links and signed direct-to-Cloudinary uploads.
- Two-Admin lifecycle approvals and guest email notifications for guest-facing event changes.

1.2 Out of scope

- Payments, UPI or bank processing; vendor booking or vendor accounts; guest accounts; automated WhatsApp delivery; in-app notification centre; ZIP gallery downloads; AI features; multi-activity event agendas; custom per-manager permissions.

1.3 Design principles

- Wedding-specific routes include :weddingId. Authentication, invitation acceptance, public pages, and guest-link routes are outside the authenticated wedding route namespace.
- Every wedding-scoped request must validate the active membership and verify that referenced records belong to the same wedding.
- The API uses explicit, narrow response DTOs. Never serialize Mongoose documents directly to the client.
- Sensitive bearer tokens are returned only when a link is first generated; only hashes are stored.
- Business invariants are enforced in backend services and reinforced by MongoDB indexes/transactions where applicable.

2. API conventions

2.1 Base path and transport

```
https://<api-host>/api/v1
```

Production traffic must use HTTPS. The API accepts and returns JSON except CSV import preview, which uses multipart/form-data or a documented JSON row payload. The preferred MVP import contract is JSON rows after client-side CSV parsing, so the server can validate and preview without needing a file parser in the upload path.

2.2 HTTP methods

Method	Usage
GET	Read a resource or collection; no state change.
POST	Create a resource, start an operation, accept an invitation, or create an upload authorization.
PATCH	Partially update editable fields.
PUT	Replace a defined relationship set, such as the event invitations assigned to a guest family.
DELETE	Soft-delete or revoke a specifically permitted resource. Permanent wedding deletion is not supported.

2.3 Authentication headers and cookies

```
Authorization: Bearer <accessToken>
Content-Type: application/json
Cookie: weddingos_refresh=<refreshToken>
```

- Access token: JWT, 15-minute lifetime. Sent in the Authorization header; keep it in memory in the frontend where practical.
- Refresh token: JWT with session identifier and token identifier, seven-day sliding inactivity expiry; sent only in a Secure, HttpOnly cookie and never in JSON.

- Refresh rotation: every successful refresh issues a new refresh token and replaces the stored hash. Reuse of an old rotated token revokes the affected session.
- Multiple sessions are allowed. Logout revokes the current session; logout-all revokes all sessions for the user.
- Cookie deployment: prefer same-site API routing. If frontend and API are cross-site, use SameSite=None; Secure, credentialed CORS, strict Origin checks and CSRF protection on cookie-authenticated endpoints.

2.4 Standard success envelope

```
{
  "success": true,
  "message": "Events retrieved successfully.",
  "data": {},
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 46,
    "totalPages": 3
  }
}
```

The pagination object is included only for paginated collection responses. For non-paginated responses, omit it. Do not return unused top-level keys as null.

2.5 Standard error envelope

```
{
  "success": false,
  "error": {
    "code": "WEDDING_ACCESS_DENIED",
    "message": "You do not have access to this wedding.",
    "details": [],
    "requestId": "req_..."
  }
}
```

2.6 Pagination, filtering and sorting

Parameter	Rule
page	1-based integer; default 1.
limit	Default 20; minimum 1; maximum 100.
sort	Allowlisted fields only; default newest/updated first as appropriate.
q	Optional bounded text search where documented; no unbounded regex.
status / categoryId / eventId / dateFrom / dateTo	Optional allowlisted filters; validate ObjectIds and ISO date-time values.

Offset pagination is the MVP standard, matching the approved SDD. For large collections, cursor pagination can be introduced in a future API version without changing resource ownership rules.

2.7 Date, time and money formats

- Send date-time values as ISO 8601 strings with an explicit timezone offset or Z. Persist instants in UTC and render using the wedding timezone (default Asia/Kolkata).
- Use integer paise for all INR amounts. Example: ₹80,000 is amountPaise: 8000000. Never use floating-point amounts.
- Use ISO 4217 currency codes; MVP currency is INR unless product requirements are expanded.

3. Authentication and authorization model

3.1 Actor types

Actor	Authentication	Scope
User	JWT access token	Account-level endpoints and active wedding memberships.
Admin	JWT + active ADMIN membership	Full wedding control, including Admin invitations, member removal, expense recycle bin, gallery deletion, lifecycle approvals.
Manager	JWT + active MANAGER membership	Operational access: events, guests, categories, expenses, vendors, tasks, gallery view/upload; cannot manage Admins/lifecycle or delete expenses/gallery files.
Guest family	Opaque family invitation token	Can view only the family's invited events and submit/update that family's RSVP.
Gallery link holder	Opaque view or upload token	Can view/download gallery files or upload, depending on the link. Links are bearer credentials and may be forwarded.
Public visitor	Unlisted public-page token	Can view the public wedding landing-page payload only.

3.2 Membership checks

- For every /weddings/:weddingId/* request, validate that the authenticated user has an ACTIVE weddingMembers record for that wedding.
- Do not trust a role claim in a long-lived JWT as the sole authorization source. Read the current membership state, or use a cache with reliable invalidation on member removal.
- A removed Manager loses access to that wedding immediately but remains logged into their global account and other weddings.
- For every referenced record, validate both _id and weddingId. A valid ObjectId alone is not proof of authorization.

3.3 Invitation model

- Use one weddingInvitations collection with invitedRole = ADMIN or MANAGER.
- Invitation token is random, unpredictable, single-use, stored only as a hash, and expires seven days after creation.
- Only an Admin can invite an Admin. Admins and Managers can invite Managers. A wedding can have no more than two active Admins.
- Invitee must be logged in with a verified email matching emailNormalized on the invitation.
- Manual invitation revocation is not exposed in v1.0. Expired or accepted invitations are not reusable.

4. Role and lifecycle permission matrix

Capability	Admin	Manager	Guest / link holder
View and edit basic wedding details	Yes	Yes	Public page only
Invite/remove Managers	Yes	Invite only	No
Invite second Admin	Yes	No	No

Capability	Admin	Manager	Guest / link holder
Manage events and guest-facing event details	Yes	Yes	View invited events only
Manage guest families and event invitations	Yes	Yes	No
Submit/update family RSVP	No special action	No special action	Own family only
Create/edit expenses and record payments	Yes	Yes	No
Delete/restore expenses and void payments	Yes	No	No
Manage saved vendors	Yes	Yes	No
Create/assign/update/delete tasks	Yes	Yes	No
View/upload gallery files	Yes	Yes	View/upload per link
Delete gallery files	Yes	No	No
Initiate/approve lifecycle transitions	Yes (two distinct Admins)	No	No
Upload gallery while Completed	Yes	Yes	Yes, via upload link
Upload gallery while Archived	No	No	No

All Admins are equal. The wedding creator is the first Admin; the second Admin must accept a dedicated Admin invitation. Admin invitation count is capped at two active Admin memberships per wedding.

4.1 Lifecycle write policy

Lifecycle	Operational writes	Gallery uploads	Read access
ACTIVE	Allowed, subject to role	Allowed	Members and permitted guests
COMPLETED	Blocked; workspace read-only	Allowed	Members and permitted guests
ARCHIVED	Blocked	Blocked	Admins/Managers; guest links and galleries remain viewable

Guest RSVP updates are accepted only while the wedding is ACTIVE. Completed and Archived weddings retain invitation and gallery viewing access, but guest RSVP modification is disabled.

5. Endpoint catalogue

The catalogue below is the complete planned v1.0 route surface. Detailed contracts follow in Section 6. All route parameters are URL-encoded. {weddingId}, {eventId}, {guestFamilyId}, and other IDs refer to MongoDB ObjectIds unless explicitly identified as opaque tokens.

Module	Method	Path (after /api/v1)	Access
Auth	POST	/auth/register	Public
Auth	POST	/auth/login	Public
Auth	POST	/auth/google	Public
Auth	POST	/auth/verify-email	Public token
Auth	POST	/auth/resend-verification	Public
Auth	POST	/auth/refresh	Refresh cookie
Auth	POST	/auth/logout	Refresh cookie

Module	Method	Path (after /api/v1)	Access
Auth	POST	/auth/logout-all	User
Auth	GET	/auth/me	User
Auth	GET	/auth/sessions	User
Auth	DELETE	/auth/sessions/:sessionId	User
Auth	POST	/auth/forgot-password	Public
Auth	POST	/auth/reset-password	Public token
Auth	POST	/auth/change-password	User
Weddings	GET	/weddings	User
Weddings	POST	/weddings	User
Weddings	GET	/weddings/:weddingId	Member
Weddings	PATCH	/weddings/:weddingId	Admin/Manager
Lifecycle	POST	/weddings/:weddingId/lifecycle-requests	Admin
Lifecycle	POST	/weddings/:weddingId/lifecycle-requests/:requestId/approve	Other Admin
Lifecycle	DELETE	/weddings/:weddingId/lifecycle-requests/:requestId	Initiating Admin
Members	GET	/weddings/:weddingId/members	Admin/Manager
Invitations	POST	/weddings/:weddingId/invitations	Admin/Manager, role constrained
Invitations	GET	/weddings/:weddingId/invitations	Admin/Manager
Invitations	GET	/public/invitations/:token	Public token
Invitations	POST	/invitations/accept	User
Members	DELETE	/weddings/:weddingId/members/:memberId	Admin
Events	GET	/weddings/:weddingId/events	Admin/Manager
Events	POST	/weddings/:weddingId/events	Admin/Manager
Events	GET	/weddings/:weddingId/events/:eventId	Admin/Manager
Events	PATCH	/weddings/:weddingId/events/:eventId	Admin/Manager
Categories	GET	/weddings/:weddingId/categories	Admin/Manager
Categories	POST	/weddings/:weddingId/categories	Admin/Manager
Categories	PATCH	/weddings/:weddingId/categories/:categoryId	Admin/Manager
Categories	DELETE	/weddings/:weddingId/categories/:categoryId	Admin/Manager (archive)
Guests	GET	/weddings/:weddingId/guest-families	Admin/Manager
Guests	POST	/weddings/:weddingId/guest-families	Admin/Manager
Guests	GET	/weddings/:weddingId/guest-families/:guestFamilyId	Admin/Manager
Guests	PATCH	/weddings/:weddingId/guest-families/:guestFamilyId	Admin/Manager
Guests	DELETE	/weddings/:weddingId/guest-families/:guestFamilyId	Admin/Manager
Guests	POST	/weddings/:weddingId/guest-families/import/preview	Admin/Manager
Guests	POST	/weddings/:weddingId/guest-families/import/commit	Admin/Manager
Guests	GET	/weddings/:weddingId/guest-families/:guestFamilyId/event-invitations	Admin/Manager
Guests	PUT	/weddings/:weddingId/guest-families/:guestFamilyId/event-invitations	Admin/Manager

Module	Method	Path (after /api/v1)	Access
Guest public	GET	/public/guest-invitations/:token	Guest token
Guest public	PUT	/public/guest-invitations/:token/rsvps/:eventId	Guest token
Expenses	GET	/weddings/:weddingId/expenses	Admin/Manager
Expenses	POST	/weddings/:weddingId/expenses	Admin/Manager
Expenses	GET	/weddings/:weddingId/expenses/:expenseId	Admin/Manager
Expenses	PATCH	/weddings/:weddingId/expenses/:expenseId	Admin/Manager
Expenses	DELETE	/weddings/:weddingId/expenses/:expenseId	Admin
Expenses	GET	/weddings/:weddingId/expenses/recycle-bin	Admin
Expenses	POST	/weddings/:weddingId/expenses/:expenseId/restore	Admin
Payments	GET	/weddings/:weddingId/expenses/:expenseId/payments	Admin/Manager
Payments	POST	/weddings/:weddingId/expenses/:expenseId/payments	Admin/Manager
Payments	POST	/weddings/:weddingId/expenses/:expenseId/payments/:paymentId/void	Admin
Vendors	GET	/weddings/:weddingId/vendors	Admin/Manager
Vendors	POST	/weddings/:weddingId/vendors	Admin/Manager
Vendors	PATCH	/weddings/:weddingId/vendors/:vendorId	Admin/Manager
Vendors	DELETE	/weddings/:weddingId/vendors/:vendorId	Admin/Manager (soft remove)
Directory	GET	/vendor-directory	Public/provider
Tasks	GET	/weddings/:weddingId/tasks	Admin/Manager
Tasks	POST	/weddings/:weddingId/tasks	Admin/Manager
Tasks	GET	/weddings/:weddingId/tasks/:taskId	Admin/Manager
Tasks	PATCH	/weddings/:weddingId/tasks/:taskId	Admin/Manager
Tasks	DELETE	/weddings/:weddingId/tasks/:taskId	Admin/Manager
Galleries	GET	/weddings/:weddingId/galleries	Admin/Manager
Galleries	GET	/weddings/:weddingId/galleries/:galleryId	Admin/Manager
Galleries	POST	/weddings/:weddingId/galleries/:galleryId/links/rotate	Admin
Galleries	POST	/weddings/:weddingId/galleries/:galleryId/uploads/signature	Admin/Manager
Galleries	POST	/weddings/:weddingId/galleries/:galleryId/uploads/complete	Admin/Manager
Galleries	DELETE	/weddings/:weddingId/gallery-files/:fileId	Admin
Galleries	POST	/weddings/:weddingId/gallery-files/bulk-delete	Admin
Gallery public	GET	/public/galleries/:viewToken	View token
Gallery public	POST	/public/galleries/:uploadToken/uploads/signature	Upload token
Gallery public	POST	/public/galleries/:uploadToken/uploads/complete	Upload token
Dashboard	GET	/weddings/:weddingId/dashboard	Admin/Manager

Module	Method	Path (after /api/v1)	Access
Public page	GET	/public/weddings/:token	Public token
Public page	POST	/weddings/:weddingId/public-page-link/rotate	Admin

6. Detailed endpoint contracts

The following endpoint definitions are normative unless identified as an example. Response examples abbreviate ObjectIds and tokens for readability. The implementation may add harmless metadata but must preserve the documented fields, semantics, and security constraints.

6.1 Authentication and account security

Register with email and password

Property	Specification
Method / path	POST /api/v1/auth/register
Authentication	Public
Allowed actor	Unauthenticated visitor
Success status	201 Created
Purpose	Create a global user account and queue an email-verification message.

Request

```
{
  "name": "Aditya Mishra",
  "email": "aditya@example.com",
  "password": "StrongPassword123"
}
```

Example response

```
{
  "success": true,
  "message": "Registration successful. Please verify your email.",
  "data": { "userId": "USER_ID", "email": "aditya@example.com" }
}
```

Rules

- Normalize email before lookup and persistence.
- Store a strong password hash (bcrypt per SDD), never a plaintext password.
- Do not issue a session until email verification succeeds.
- Return 409 EMAIL_ALREADY_REGISTERED for an existing normalized email.

Login

Property	Specification
Method / path	POST /api/v1/auth/login
Authentication	Public
Allowed actor	Verified account holder
Success status	200 OK
Purpose	Authenticate with email/password and create a device session.

Request

```
{
  "email": "aditya@example.com",
  "password": "StrongPassword123"
}
```

Example response

```
{
  "success": true,
  "data": {
    "accessToken": "JWT_ACCESS_TOKEN",
    "expiresIn": 900,
    "user": { "id": "USER_ID", "name": "Aditya Mishra", "email": "aditya@example.com" }
  }
}
```

Rules

- Set the refresh token in a Secure, HttpOnly cookie. Do not include it in JSON.
- Reject unverified email/password accounts with 403 EMAIL_NOT_VERIFIED.
- Use generic invalid-credentials messaging to avoid account enumeration.
- Record session metadata such as user-agent and last-used timestamp.

Google sign-in / sign-up

Property	Specification
Method / path	POST /api/v1/auth/google
Authentication	Public
Allowed actor	Google-authenticated visitor
Success status	200 OK
Purpose	Verify a Google ID token, sign in an existing account, link an exact matching verified email, or create a new account.

Request

```
{ "idToken": "GOOGLE_ID_TOKEN" }
```

Example response

```
{
  "success": true,
  "data": {
    "accessToken": "JWT_ACCESS_TOKEN",
    "expiresIn": 900,
    "user": { "id": "USER_ID", "name": "Aditya Mishra", "email": "aditya@gmail.com" }
  }
}
```

Rules

- Verify signature, issuer, audience, expiry, subject and email_verified claim server-side.
- Use Google subject (sub) as the provider identity; never trust client-decoded token fields.
- If a verified Google email matches an existing account, link the Google identity to that account rather than creating a duplicate.
- Create a new user with verified email when no account exists.

Verify email and auto-login

Property	Specification
Method / path	POST /api/v1/auth/verify-email
Authentication	Public verification token

Property	Specification
Allowed actor	New registrant with a valid verification link
Success status	200 OK
Purpose	Consume a one-time email verification token, mark the email verified, and create an authenticated session.

Request

```
{ "token": "EMAIL_VERIFICATION_TOKEN" }
```

Example response

```
{
  "success": true,
  "message": "Email verified successfully.",
  "data": {
    "accessToken": "JWT_ACCESS_TOKEN",
    "expiresIn": 900,
    "user": { "id": "USER_ID", "emailVerified": true }
  }
}
```

Rules

- Set refresh token in HttpOnly cookie.
- Token must be single-use and expire; store only its hash.
- Verification token should be consumed atomically with emailVerifiedAt update and session creation.

Resend verification

Property	Specification
Method / path	POST /api/v1/auth/resend-verification
Authentication	Public
Allowed actor	Unverified registrant
Success status	200 OK
Purpose	Queue another verification email.

Request

```
{ "email": "aditya@example.com" }
```

Example response

```
{
  "success": true,
  "message": "If verification is pending for this email, a new link will be sent."
}
```

Rules

- Return a generic response.
- Apply per-email and per-IP rate limits.
- A newly issued token supersedes earlier unconsumed verification tokens.

Refresh access token

Property	Specification
Method / path	POST /api/v1/auth/refresh
Authentication	Refresh cookie + CSRF/Origin protection
Allowed actor	User with a valid active session cookie

Property	Specification
Success status	200 OK
Purpose	Rotate the refresh token and issue a new access token.

Example response

```
{
  "success": true,
  "data": { "accessToken": "NEW_ACCESS_TOKEN", "expiresIn": 900 }
}
```

Rules

- Refresh token lifetime is seven days of inactivity.
- Rotate on every successful refresh and replace the stored token hash.
- Detect reuse of a previously rotated token and revoke that session family.
- Set the replacement refresh cookie; never return it in JSON.

Logout current session

Property	Specification
Method / path	POST /api/v1/auth/logout
Authentication	Refresh cookie
Allowed actor	Authenticated session
Success status	200 OK
Purpose	Revoke the current device session and clear its refresh cookie.

Example response

```
{ "success": true, "message": "Logged out successfully." }
```

Rules

- Revocation is idempotent.
- Clear the refresh cookie using the same path, domain and security attributes used to set it.

Logout all devices

Property	Specification
Method / path	POST /api/v1/auth/logout-all
Authentication	Access token
Allowed actor	Logged-in user
Success status	200 OK
Purpose	Revoke all active sessions for the user.

Example response

```
{ "success": true, "message": "All sessions have been revoked." }
```

Rules

- Also clear the current refresh cookie.
- Other access tokens may remain valid for their remaining 15-minute lifetime unless a separate access-token denylist is implemented; refresh is immediately blocked.

Get current user

Property	Specification
Method / path	GET /api/v1/auth/me
Authentication	Access token
Allowed actor	Logged-in user
Success status	200 OK
Purpose	Return safe account profile and authentication state.

Example response

```
{
  "success": true,
  "data": { "id": "USER_ID", "name": "Aditya Mishra", "email": "aditya@example.com", "emailVerified": true }
}
```

Rules

- Never return passwordHash, Google secrets, token hashes or session secrets.

List device sessions

Property	Specification
Method / path	GET /api/v1/auth/sessions
Authentication	Access token
Allowed actor	Logged-in user
Success status	200 OK
Purpose	List active sessions belonging to the current account.

Example response

```
{
  "success": true,
  "data": [{ "id": "SESSION_ID", "device": "Chrome on Windows", "createdAt": "2026-11-09T05:00:00Z", "lastUsedAt": "2026-11-09T05:30:00Z",
"expiresAt": "2026-11-16T05:30:00Z", "current": true }]
}
```

Rules

- Device label is derived from sanitized user-agent metadata and is informational only.
- Do not return refresh-token hashes.

Revoke one device session

Property	Specification
Method / path	DELETE /api/v1/auth/sessions/:sessionId
Authentication	Access token
Allowed actor	Session owner
Success status	200 OK
Purpose	Revoke a selected active session.

Example response

```
{ "success": true, "message": "Session revoked." }
```

Rules

- A user may revoke only sessions belonging to their own account.
- Clear the cookie if the selected session is the current session.

Request password reset

Property	Specification
Method / path	POST /api/v1/auth/forgot-password
Authentication	Public
Allowed actor	Unauthenticated visitor
Success status	200 OK
Purpose	Send a reset link if an account exists.

Request

```
{ "email": "aditya@example.com" }
```

Example response

```
{
  "success": true,
  "message": "If an account exists with this email, a password reset link will be sent."
}
```

Rules

- Always return the same response whether or not the email exists.
- Reset tokens expire after 30 minutes, are single-use, random and stored as hashes.
- Rate-limit requests by IP and normalized email.

Reset password

Property	Specification
Method / path	POST /api/v1/auth/reset-password
Authentication	Public reset token
Allowed actor	User with a valid reset link
Success status	200 OK
Purpose	Set a new password and revoke all existing sessions.

Request

```
{
  "token": "PASSWORD_RESET_TOKEN",
  "newPassword": "NewStrongPassword123"
}
```

Example response

```
{ "success": true, "message": "Password reset successfully. Please log in again." }
```

Rules

- Consume reset token atomically.
- Revoke all authSessions for the user after password update.
- Do not automatically log in after password reset.

Change password

Property	Specification
Method / path	POST /api/v1/auth/change-password

Property	Specification
Authentication	Access token
Allowed actor	Logged-in email/password account
Success status	200 OK
Purpose	Change password while authenticated.

Request

```
{
 "currentPassword": "OldPassword123",
 "newPassword": "NewStrongPassword123"
}
```

Example response

```
{ "success": true, "message": "Password changed successfully." }
```

Rules

- Require current password.
- Revoke all sessions except optionally issue a fresh current session; v1.0 default is revoke all and require login again.
- For Google-only accounts without passwordHash, return 409 PASSWORD_NOT_CONFIGURED.

6.2 Weddings and lifecycle

List accessible weddings

Property	Specification
Method / path	GET /api/v1/weddings
Authentication	Access token
Allowed actor	Any logged-in user
Success status	200 OK
Purpose	Return all weddings where the user has an ACTIVE membership, including ACTIVE, COMPLETED and ARCHIVED workspaces.

Example response

```
{
 "success": true,
 "data": [{ "weddingId": "WEDDING_ID", "brideName": "Bride", "groomName": "Groom", "role": "ADMIN", "lifecycle": "ACTIVE", "weddingDate":
null }],
 "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

Rules

- Sort by updatedAt descending by default.
- Include membership role; do not infer ownership from role alone.
- Removed memberships are excluded.

Create wedding

Property	Specification
Method / path	POST /api/v1/weddings
Authentication	Access token; email verified
Allowed actor	User who has not already created a wedding

Property	Specification
Success status	201 Created
Purpose	Create a wedding and creator Admin membership in one MongoDB transaction.

Request

```
{
  "brideName": "Bride Name",
  "groomName": "Groom Name",
  "weddingDate": null,
  "venue": { "name": null, "address": null, "mapUrl": null }
}
```

Example response

```
{
  "success": true,
  "data": {
    "wedding": { "id": "WEDDING_ID", "brideName": "Bride Name", "groomName": "Groom Name", "lifecycle": "ACTIVE" },
    "membership": { "role": "ADMIN", "status": "ACTIVE" }
  }
}
```

Rules

- Enforce one created wedding per user using a unique index on weddings.createdBy.
- Create wedding + initial ADMIN membership + public page token atomically.
- Second Admin joins only through an accepted ADMIN invitation.
- Return 409 WEDDING_ALREADY_CREATED when the user has already created one.

Get wedding details

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Return editable workspace details and lifecycle state.

Example response

```
{
  "success": true,
  "data": { "id": "WEDDING_ID", "brideName": "Bride", "groomName": "Groom", "weddingDate": null, "venue": {}, "timezone": "Asia/Kolkata",
"lifecycle": "ACTIVE" }
}
```

Rules

- Exclude public page token hashes, invitation tokens and internal secrets.

Update wedding details

Property	Specification
Method / path	PATCH /api/v1/weddings/:weddingId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK

Property	Specification
Purpose	Update bride/groom display names, optional date and main venue.

Request

```
{
  "brideName": "Updated Bride Name",
  "weddingDate": "2027-02-15T18:30:00+05:30",
  "venue": { "name": "Grand Hall", "address": "Kanpur", "mapUrl": "https://maps.google.com/..." }
}
```

Example response

```
{ "success": true, "data": { "id": "WEDDING_ID", "updatedAt": "2026-11-09T06:00:00Z" } }
```

Rules

- Do not allow lifecycle, createdBy, membership roles or system-managed tokens to be patched here.
- Allowed in ACTIVE only.
- Validate venue URL and date fields.

Initiate lifecycle request

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/lifecycle-requests
Authentication	Access token + active ADMIN membership
Allowed actor	Admin
Success status	201 Created
Purpose	Start a two-Admin request to complete, reopen, archive or restore a wedding.

Request

```
{ "action": "COMPLETE" }
```

Example response

```
{
  "success": true,
  "data": { "requestId": "REQUEST_ID", "action": "COMPLETE", "status": "PENDING", "initiatedBy": "ADMIN_1", "approvalCount": 1,
"requiredApprovals": 2 }
}
```

Rules

- Supported transitions: ACTIVE -> COMPLETED; COMPLETED -> ACTIVE (REOPEN); COMPLETED -> ARCHIVED; ARCHIVED -> COMPLETED (RESTORE).
- Initiating Admin counts as the first approval. The other active Admin must approve.
- Require exactly two active Admin memberships; otherwise return 409 SECOND_ADMIN_REQUIRED.
- Only one pending lifecycle request per wedding.

Approve lifecycle request

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/lifecycle-requests/:requestId/approve
Authentication	Access token + active ADMIN membership
Allowed actor	The other Admin, not the initiator
Success status	200 OK

Property	Specification
Purpose	Record the second Admin approval and apply the transition.

Request

```
{ "confirmation": true }
```

Example response

```
{
  "success": true,
  "data": { "lifecycle": "COMPLETED", "requestStatus": "APPROVED", "approvedBy": ["ADMIN_1", "ADMIN_2"] }
}
```

Rules

- Approver must be distinct from initiator and be an active Admin at approval time.
- Use a transaction to record approval, update lifecycle and system-complete PENDING tasks on completion.
- On archive, block future gallery uploads. On restore, return to COMPLETED; a separate REOPEN request is required to return to ACTIVE.

Cancel pending lifecycle request

Property	Specification
Method / path	DELETE /api/v1/weddings/:weddingId/lifecycle-requests/:requestId
Authentication	Access token + active ADMIN membership
Allowed actor	Initiating Admin
Success status	200 OK
Purpose	Cancel a pending request before the second Admin approves.

Example response

```
{ "success": true, "message": "Lifecycle request cancelled." }
```

Rules

- Only the initiating Admin may cancel.
- Cancellation does not change wedding lifecycle.

6.3 Membership and invitations

List wedding members

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/members
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List active and removed membership records for the wedding.

Example response

```
{
  "success": true,
  "data": [{ "memberId": "MEMBER_ID", "user": { "id": "USER_ID", "name": "Rahul", "email": "rahul@example.com" }, "role": "MANAGER", "status":
"ACTIVE", "joinedAt": "2026-11-01T00:00:00Z" }]
}
```

Rules

- Return only safe user profile fields.
- Do not expose auth session details.

Create Admin or Manager invitation

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/invitations
Authentication	Access token + active membership
Allowed actor	ADMIN may invite ADMIN or MANAGER; MANAGER may invite MANAGER only
Success status	201 Created
Purpose	Create a seven-day single-use invitation, email it, and return a shareable link once.

Request

```
{
 "email": "manager@example.com",
 "role": "MANAGER"
}
```

Example response

```
{
 "success": true,
 "data": { "invitationId": "INVITATION_ID", "role": "MANAGER", "expiresAt": "2026-11-16T06:00:00Z", "invitationUrl": "https://.../invite/<raw-token>" }
}
```

Rules

- Normalize invited email.
- Only ADMIN may set role=ADMIN; at most two active ADMIN memberships.
- Create a single-use token; store only tokenHash. Expiry is seven days.
- Invitation cannot be manually revoked in v1.0.
- Return raw invitation URL only on creation; it must not be logged.
- If a pending invitation exists for same wedding/email/role, return 409 INVITATION_ALREADY_PENDING or issue a fresh token under a documented resend policy.

List invitations

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/invitations
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List pending, accepted and expired invitations without exposing token values.

Example response

```
{
 "success": true,
 "data": [{ "id": "INVITATION_ID", "email": "manager@example.com", "role": "MANAGER", "status": "PENDING", "expiresAt": "2026-11-
16T06:00:00Z" }]
}
```

Rules

- Never return tokenHash or raw token.

Preview invitation

Property	Specification
Method / path	GET /api/v1/public/invitations/:token
Authentication	Public opaque token
Allowed actor	Invitation link holder
Success status	200 OK
Purpose	Validate a link and show a safe invitation preview before login/acceptance.

Example response

```
{
  "success": true,
  "data": { "wedding": { "brideName": "Bride", "groomName": "Groom" }, "invitedRole": "MANAGER", "invitedEmailMasked": "m***@example.com",
"expiresAt": "2026-11-16T06:00:00Z", "status": "PENDING" }
}
```

Rules

- Do not reveal the full invited email to an unauthenticated visitor.
- Invalid, expired or used tokens return the same generic invalid-link response.

Accept invitation

Property	Specification
Method / path	POST /api/v1/invitations/accept
Authentication	Access token + verified email
Allowed actor	Authenticated invitee whose normalized verified email matches invite
Success status	200 OK
Purpose	Accept a pending invitation and create/reactivate the wedding membership transactionally.

Request

```
{ "token": "INVITATION_TOKEN" }
```

Example response

```
{
  "success": true,
  "data": { "weddingId": "WEDDING_ID", "role": "MANAGER", "membershipStatus": "ACTIVE" }
}
```

Rules

- Validate invitation token, expiry, role capacity and exact verified email match.
- Use a transaction to create membership and mark invitation ACCEPTED.
- A previously removed member may be reactivated only if the invitation is valid and the role rules permit it.
- Unique {userId, weddingId} membership index prevents duplicates.

Remove Manager

Property	Specification
Method / path	DELETE /api/v1/weddings/:weddingId/members/:memberId

Property	Specification
Authentication	Access token + active ADMIN membership
Allowed actor	Admin only
Success status	200 OK
Purpose	Remove an active Manager from the wedding.

Example response

```
{ "success": true, "message": "Manager removed from this wedding." }
```

Rules

- Cannot remove either Admin through this endpoint.
- If Manager has pending or in-progress assigned tasks, return 409 MANAGER_HAS_OPEN_TASKS; Admin must reassign those tasks first.
- Mark membership REMOVED and revoke wedding access immediately. Do not revoke the user global sessions.

6.4 Events

List events

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/events
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List wedding events with safe member-visible details.

Example response

```
{
  "success": true,
  "data": [{ "id": "EVENT_ID", "name": "Mehendi", "status": "SCHEDULED", "guestInfo": { "startAt": "2027-02-14T10:00:00+05:30", "endAt": "2027-02-14T14:00:00+05:30" } }],
  "pagination": { "page": 1, "limit": 20, "total": 4, "totalPages": 1 }
}
```

Rules

- Support filters status, dateFrom, dateTo and bounded q search.
- Internal details may be included for members but never in guest/public DTOs.

Create event

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/events
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	201 Created
Purpose	Create a scheduled event and its event gallery.

Request

```
{
  "name": "Mehendi",
  "guestInfo": {
    "description": "Mehendi ceremony",
    "startAt": "2027-02-14T10:00:00+05:30",
    "endAt": "2027-02-14T14:00:00+05:30",
    "venue": { "name": "Grand Hall", "address": "Kanpur", "mapUrl": "https://maps.google.com/..." },
    "dressCode": "Traditional"
  },
  "internalInfo": { "notes": "Coordinate stage setup", "assignedManagerIds": ["USER_ID"] }
}
```

Example response

```
{
  "success": true,
  "data": {
    "event": { "id": "EVENT_ID", "name": "Mehendi", "status": "SCHEDULED" },
    "gallery": { "id": "GALLERY_ID", "viewUrl": "https://.../gallery/view/<token>", "uploadUrl": "https://.../gallery/upload/<token>" }
  }
}
```

Rules

- Create event and gallery in a transaction. Return raw gallery links only on initial generation; store hashes only.
- Validate all assignedManagerIds are active members of this wedding.
- Event must start before or at its end; support multi-day events.
- Allowed only while lifecycle is ACTIVE.

Get event

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/events/:eventId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Return one event with member-only internal information.

Example response

```
{ "success": true, "data": { "id": "EVENT_ID", "name": "Mehendi", "guestInfo": {}, "internalInfo": {}, "status": "SCHEDULED" } }
```

Rules

- Validate event weddingId.

Update event / cancel event

Property	Specification
Method / path	PATCH /api/v1/weddings/:weddingId/events/:eventId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Update event fields or mark the event CANCELLED without deleting it.

Request

```
{
 "guestInfo": { "startAt": "2027-02-14T11:00:00+05:30", "venue": { "name": "New Hall", "address": "Kanpur", "mapUrl": "https://maps.google.com/..." } },
 "internalInfo": { "notes": "Updated setup plan" },
 "status": "SCHEDULED"
}
```


Example response


```
{ "success": true, "data": { "id": "EVENT_ID", "updatedAt": "2026-11-09T06:30:00Z", "guestInfoVersion": 2 } }
```


Rules

- Only allowlisted fields may be patched.
- If guestInfo changes or status becomes CANCELLED, create emailOutbox jobs for families invited to this event.
- Changes only to internalInfo do not notify guests.
- Cancellation is a retained status, not a hard delete.
- Allowed only while lifecycle is ACTIVE.

6.5 Categories and guest families

List categories

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/categories
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List categories, optionally filtered by type=GUEST | EXPENSE | VENDOR.

Example response


```
{ "success": true, "data": [{ "id": "CATEGORY_ID", "type": "GUEST", "name": "Bride's Family", "status": "ACTIVE" }] }
```


Rules

- Archived categories are omitted by default; allow includeArchived=true for Admin/Manager views.

Create category

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/categories
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	201 Created
Purpose	Create a custom wedding-scoped category.

Request


```
{ "type": "GUEST", "name": "Bride's Family" }
```


Example response


```
{ "success": true, "data": { "id": "CATEGORY_ID", "type": "GUEST", "name": "Bride's Family" } }
```


Rules

- Normalize category name and enforce unique {weddingId,type,normalizedName}.
- Allowed only while lifecycle is ACTIVE.

Update category

Property	Specification
Method / path	PATCH /api/v1/weddings/:weddingId/categories/:categoryId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Rename an existing active category.

Request

```
{ "name": "Bride Side" }
```

Example response

```
{ "success": true, "data": { "id": "CATEGORY_ID", "name": "Bride Side" } }
```

Rules

- Category type is immutable in v1.0.
- Validate category belongs to the wedding.

Archive category

Property	Specification
Method / path	DELETE /api/v1/weddings/:weddingId/categories/:categoryId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Archive a category while preserving historical references.

Example response

```
{ "success": true, "message": "Category archived." }
```

Rules

- Do not hard-delete categories.
- Archived categories cannot be assigned to new records; existing records may continue referencing them.

List guest families

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/guest-families
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List guest-family records with category and event-invitation summary.

Example response

```
{
  "success": true,
  "data": [{ "id": "FAMILY_ID", "familyName": "Sharma Family", "primaryContact": { "name": "Rahul Sharma", "email": "rahul@example.com" },
"categoryId": "CATEGORY_ID", "status": "ACTIVE" }],
  "pagination": { "page": 1, "limit": 20, "total": 120, "totalPages": 6 }
}
```


Rules

- Support q, categoryId and status filters.
- Never return invitationTokenHash.

Create guest family

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/guest-families
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	201 Created
Purpose	Create one family-level guest record and issue its permanent invitation link.

Request


```
{
  "familyName": "Sharma Family",
  "primaryContact": { "name": "Rahul Sharma", "email": "rahul@example.com", "phone": "+91XXXXXXXXXX" },
  "categoryId": "CATEGORY_ID"
}
```


Example response


```
{
  "success": true,
  "data": { "id": "FAMILY_ID", "familyName": "Sharma Family", "status": "ACTIVE", "invitationUrl": "https://.../guest/<raw-token>" }
}
```


Rules

- Family name and primary contact name/email are required; phone is optional.
- Generate an unpredictable token; store only its hash.
- Detect possible duplicates and return warnings; do not auto-merge.
- Return raw invitation link only on creation.

Get guest family

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/guest-families/:guestFamilyId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Get family details, invited event IDs and RSVP summary.

Example response


```
{ "success": true, "data": { "id": "FAMILY_ID", "familyName": "Sharma Family", "primaryContact": {}, "categoryId": "CATEGORY_ID",
"eventInvitations": [], "rsvps": [] } }
```


Rules

- Validate weddingId on family and related invitations/RSVPs.

Update guest family

Property	Specification
Method / path	PATCH /api/v1/weddings/:weddingId/guest-families/:guestFamilyId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Edit family name, contact and category.

Request

```
{ "familyName": "Sharma Family", "primaryContact": { "name": "Rahul Sharma", "email": "rahul@example.com", "phone": "+91XXXXXXXXXX" }, "categoryId": "CATEGORY_ID" }
```

Example response

```
{ "success": true, "data": { "id": "FAMILY_ID", "updatedAt": "2026-11-09T07:00:00Z" } }
```

Rules

- Changing family contact email does not change the opaque invitation token.
- Allowed only while lifecycle is ACTIVE.

Delete guest family / invalidate link

Property	Specification
Method / path	DELETE /api/v1/weddings/:weddingId/guest-families/:guestFamilyId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Deactivate a family and immediately invalidate its guest invitation link.

Example response

```
{ "success": true, "message": "Guest family deleted and invitation link invalidated." }
```

Rules

- Use soft deletion for audit/history but unset invitationTokenHash immediately.
- Related RSVP and invitation records remain for historical integrity and are excluded from active totals.

Preview guest import

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/guest-families/import/preview
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Validate client-parsed CSV rows and return missing/invalid/possible-duplicate warnings before commit.

Request

```
{
  "rows": [
    { "rowNumber": 2, "familyName": "Sharma Family", "contactName": "Rahul Sharma", "email": "rahul@example.com", "phone": "",
"categoryName": "Bride's Family" }
  ]
}
```


Example response


```
{
  "success": true,
  "data": { "previewId": "SHORT_LIVED_PREVIEW_ID", "expiresAt": "2026-11-09T07:30:00Z", "validCount": 1, "invalidCount": 0, "possibleDuplicates": []
}
}
```


Rules

- Store preview payload in Redis with a 30-minute TTL.
- Do not create guest records during preview.
- Return row-level validation errors and duplicate candidates; do not merge automatically.

Commit guest import

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/guest-families/import/commit
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	201 Created
Purpose	Commit selected valid preview rows after duplicate decisions.

Request


```
{
  "previewId": "SHORT_LIVED_PREVIEW_ID",
  "selectedRowNumbers": [2],
  "duplicateDecisions": []
}
```


Example response


```
{ "success": true, "data": { "createdCount": 1, "skippedCount": 0, "errors": [] } }
```


Rules

- Preview must belong to the same user and wedding and must not be expired.
- Generate unique family invitation tokens for created records.
- Use a bounded transaction/batch; do not partially commit silently.

Get family event invitations

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/guest-families/:guestFamilyId/event-invitations
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List events to which the family is currently invited.

Example response


```
{ "success": true, "data": [{ "eventId": "EVENT_ID", "status": "INVITED", "invitedAt": "2026-11-09T07:00:00Z" }] }
```

Replace family event invitations

Property	Specification
Method / path	PUT /api/v1/weddings/:weddingId/guest-families/:guestFamilyId/event-invitations
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Replace the family event invitation set with the supplied event IDs.

Request

```
{ "eventIds": ["EVENT_ID_1", "EVENT_ID_2"] }
```

Example response

```
{ "success": true, "data": { "invitedEventIds": ["EVENT_ID_1", "EVENT_ID_2"] } }
```

Rules

- All event IDs must belong to the same wedding.
- Use a transaction to upsert desired invitations and mark removed ones REVOKED.
- Existing RSVP records for revoked invitations are retained but hidden from guest access and active invited-family totals.

6.6 Guest-facing invitation and RSVP APIs

Open guest invitation page

Property	Specification
Method / path	GET /api/v1/public/guest-invitations/:token
Authentication	Public opaque family token
Allowed actor	Guest family link holder
Success status	200 OK
Purpose	Return the family contact display and only the events that family is invited to.

Example response

```
{
 "success": true,
 "data": {
  "family": { "familyName": "Sharma Family", "primaryContactName": "Rahul Sharma" },
  "wedding": { "brideName": "Bride", "groomName": "Groom" },
  "events": [{ "eventId": "EVENT_ID", "name": "Wedding", "guestInfo": {}, "rsvp": { "response": "MAYBE" } }]
 }
}
```

Rules

- Do not return other families, guest lists, manager assignments, internal notes or financial information.
- Require active guest family and INVITED relationship for each event.
- Invalid/deleted/rotated tokens return 404 with a generic message.

Submit or update RSVP

Property	Specification
Method / path	PUT /api/v1/public/guest-invitations/:token/rsvps/:eventId
Authentication	Public opaque family token
Allowed actor	Guest family link holder
Success status	200 OK
Purpose	Create or update the single family RSVP for an invited event.

Request

```
{ "response": "YES" }
```

Example response

```
{ "success": true, "data": { "eventId": "EVENT_ID", "response": "YES", "respondedAt": "2026-11-09T07:05:00Z" } }
```

Rules

- Allowed values: YES, NO, MAYBE.
- Upsert by unique {weddingId, guestFamilyId, eventId}.
- Family must be actively invited to that event and wedding lifecycle must be ACTIVE.
- The guest can update their response at any time while the wedding is ACTIVE.

6.7 Expenses and payments

List expenses

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/expenses
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List active expenses with paid and outstanding totals.

Example response

```
{
  "success": true,
  "data": [{ "id": "EXPENSE_ID", "title": "Wedding Photography", "amountPaise": 8000000, "paidPaise": 2000000, "outstandingPaise": 6000000, "status": "ACTIVE" }],
  "pagination": { "page": 1, "limit": 20, "total": 10, "totalPages": 1 }
}
```

Rules

- Support categoryId, eventId, vendorId, status filters.
- Calculate paidPaise from non-voided payments for active expenses.

Create expense

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/expenses
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	201 Created

Property	Specification
Purpose	Record a new agreed expense.

Request

```
{
  "title": "Wedding Photography",
  "amountPaise": 8000000,
  "categoryId": "CATEGORY_ID",
  "eventId": "EVENT_ID",
  "vendorId": "VENDOR_ID",
  "notes": "Full-day coverage"
}
```

Example response

```
{ "success": true, "data": { "id": "EXPENSE_ID", "amountPaise": 8000000, "paidPaise": 0, "outstandingPaise": 8000000 } }
```

Rules

- amountPaise must be a positive integer.
- Validate category/event/vendor belong to the same wedding when supplied.
- The agreed amount is immutable after creation.
- Allowed only while lifecycle is ACTIVE.

Get expense

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/expenses/:expenseId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Get expense details and payment history summary.

Example response

```
{ "success": true, "data": { "id": "EXPENSE_ID", "title": "Wedding Photography", "amountPaise": 8000000, "paidPaise": 2000000, "outstandingPaise": 6000000 } }
```

Update expense descriptive fields

Property	Specification
Method / path	PATCH /api/v1/weddings/:weddingId/expenses/:expenseId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Update title, category, event, vendor, notes or receipt references without changing the agreed amount.

Request

```
{ "title": "Wedding Photography Package", "categoryId": "CATEGORY_ID", "notes": "Updated scope" }
```

Example response

```
{ "success": true, "data": { "id": "EXPENSE_ID", "updatedAt": "2026-11-09T07:10:00Z" } }
```

Rules

- Reject amountPaise or currency changes with 422 IMMUTABLE_FIELD.

- Allowed only while lifecycle is ACTIVE.

Soft-delete expense

Property	Specification
Method / path	DELETE /api/v1/weddings/:weddingId/expenses/:expenseId
Authentication	Access token + active ADMIN membership
Allowed actor	Admin only
Success status	200 OK
Purpose	Move an expense to the recycle bin while preserving its history.

Example response

```
{ "success": true, "message": "Expense moved to recycle bin." }
```

Rules

- Record deletedBy and deletedAt.
- Do not delete associated payment records.
- Deleted expense is excluded from active dashboard totals.

List expense recycle bin

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/expenses/recycle-bin
Authentication	Access token + active ADMIN membership
Allowed actor	Admin only
Success status	200 OK
Purpose	List soft-deleted expenses available for restoration.

Example response

```
{ "success": true, "data": [{ "id": "EXPENSE_ID", "title": "Wedding Photography", "deletedBy": "USER_ID", "deletedAt": "2026-11-09T07:10:00Z" }] }
```

Restore expense

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/expenses/:expenseId/restore
Authentication	Access token + active ADMIN membership
Allowed actor	Admin only
Success status	200 OK
Purpose	Restore a soft-deleted expense and include it again in totals.

Example response

```
{ "success": true, "message": "Expense restored." }
```

Rules

- Restore only if the wedding is ACTIVE.
- Preserve original createdAt, amount and payments.

List payments

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/expenses/:expenseId/payments
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List instalments recorded against an expense.

Example response

```
{ "success": true, "data": [{ "id": "PAYMENT_ID", "amountPaise": 2000000, "paidAt": "2026-11-01T00:00:00Z", "paymentMethod": "UPI", "status": "ACTIVE" }] }
```

Record payment

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/expenses/:expenseId/payments
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	201 Created
Purpose	Record a payment instalment against an expense.

Request

```
{
  "amountPaise": 2000000,
  "paidAt": "2026-11-01T12:00:00+05:30",
  "paymentMethod": "UPI",
  "receipt": { "publicId": "cloudinary-id", "secureUrl": "https://..." }
}
```

Example response

```
{ "success": true, "data": { "id": "PAYMENT_ID", "amountPaise": 2000000, "paidPaise": 2000000, "outstandingPaise": 6000000 } }
```

Rules

- Require Idempotency-Key header for payment creation.
- Reject payment amount greater than current outstanding amount.
- Payment records are append-only; they are not edited or silently deleted.
- Allowed only while lifecycle is ACTIVE.

Void payment

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/expenses/:expenseId/payments/:paymentId/void
Authentication	Access token + active ADMIN membership
Allowed actor	Admin only
Success status	200 OK
Purpose	Void an incorrectly recorded payment without erasing the history.

Request

{ "reason": "Duplicate entry" }

Example response

{ "success": true, "data": { "paymentId": "PAYMENT_ID", "status": "VOIDED", "voidedAt": "2026-11-09T07:20:00Z" } }

Rules

- Require a non-empty reason.
- Store voidedBy and voidedAt; exclude voided payment from paid totals.
- Allowed only while lifecycle is ACTIVE.

6.8 Vendors and vendor directory

List saved vendors

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/vendors
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List wedding-saved vendors.

Example response

{ "success": true, "data": [{ "id": "VENDOR_ID", "businessName": "Royal Photography", "categoryId": "CATEGORY_ID", "status": "SHORTLISTED", "recordStatus": "ACTIVE" }] }

Rules

- Support categoryId, status and q filters.
- Exclude removed vendors by default.

Save vendor manually

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/vendors
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	201 Created
Purpose	Create a manually entered wedding vendor record.

Request

{
"businessName": "Royal Photography",
"contactPerson": "Amit",
"phone": "+91XXXXXXXXXX",
"email": "amit@example.com",
"address": "Kanpur, Uttar Pradesh",
"categoryId": "CATEGORY_ID",
"quotedAmountPaise": 9000000,
"status": "SHORTLISTED",
"notes": "Wedding package"
}

Example response

{ "success": true, "data": { "id": "VENDOR_ID", "businessName": "Royal Photography", "status": "SHORTLISTED" } }

Rules

- Vendor statuses: SHORTLISTED, CONTACTED, FINALISED, COMPLETED.
- Final agreed amount and payment totals live in linked expense/payment records, not duplicated vendor fields.

Update vendor

Property	Specification
Method / path	PATCH /api/v1/weddings/:weddingId/vendors/:vendorId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Edit saved vendor contact, quotation, category, status and notes.

Request

```
{ "status": "FINALISED", "quotedAmountPaise": 9000000, "notes": "Quote confirmed" }
```

Example response

```
{ "success": true, "data": { "id": "VENDOR_ID", "status": "FINALISED" } }
```

Rules

- Validate category belongs to wedding.
- Do not duplicate or overwrite linked expense/payment history.

Remove vendor

Property	Specification
Method / path	DELETE /api/v1/weddings/:weddingId/vendors/:vendorId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Soft-remove a vendor from active lists while preserving expense references.

Example response

```
{ "success": true, "message": "Vendor removed from saved vendors." }
```

Rules

- Preserve vendor document for linked historical expenses.
- Do not cascade-delete expense records.

Search public vendor directory

Property	Specification
Method / path	GET /api/v1/vendor-directory
Authentication	Public/provider adapter
Allowed actor	Any visitor or member
Success status	200 OK
Purpose	Search external/public business listings by category and manually entered city.

Example response

```
{
 "success": true,
 "data": [{ "name": "Example Vendor", "category": "PHOTOGRAPHER", "address": "Kanpur", "phone": "...", "website": "...", "source": "PROVIDER" }]
}
```

Rules

- Provider integration is a replaceable adapter; exact data provider is deferred.
- Do not imply listings are verified or endorsed.
- Apply provider-specific quota and cache controls.

6.9 Tasks

List tasks

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/tasks
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List event-linked tasks with optional filters.

Example response

```
{ "success": true, "data": [{ "id": "TASK_ID", "eventId": "EVENT_ID", "title": "Confirm decoration", "status": "PENDING", "assigneeUserIds": ["USER_ID"] }] }
```

Rules

- Support eventId, status, assigneeUserId and dueDate filters.

Create task

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/tasks
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	201 Created
Purpose	Create an event-related task and assign one or more active wedding members.

Request

```
{
 "eventId": "EVENT_ID",
 "title": "Confirm decoration",
 "description": "Finalise stage decoration",
 "dueDate": "2027-02-10T18:00:00+05:30",
 "assigneeUserIds": ["USER_ID_1", "USER_ID_2"]
}
```

Example response

```
{ "success": true, "data": { "id": "TASK_ID", "status": "PENDING", "assigneeUserIds": ["USER_ID_1", "USER_ID_2"] } }
```

Rules

- eventId is required and must belong to the wedding.
- Every assignee must be an active Admin or Manager in the wedding.

- Statuses: PENDING, IN_PROGRESS, COMPLETED.
- Allowed only while lifecycle is ACTIVE.

Get task

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/tasks/:taskId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Get task details.

Example response

```
{ "success": true, "data": { "id": "TASK_ID", "eventId": "EVENT_ID", "title": "Confirm decoration", "status": "PENDING" } }
```

Update task

Property	Specification
Method / path	PATCH /api/v1/weddings/:weddingId/tasks/:taskId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Update title, description, due date, assignees or status.

Request

```
{ "status": "IN_PROGRESS", "assigneeUserIds": ["USER_ID_2"] }
```

Example response

```
{ "success": true, "data": { "id": "TASK_ID", "status": "IN_PROGRESS", "updatedAt": "2026-11-09T07:30:00Z" } }
```

Rules

- Validate assignees remain active members of the same wedding.
- Any one assigned user may mark the task COMPLETED; Admins/Managers may manage assignment and other status changes.
- Do not permit a Manager removal while open tasks remain assigned to that Manager.
- Allowed only while lifecycle is ACTIVE.

Delete task

Property	Specification
Method / path	DELETE /api/v1/weddings/:weddingId/tasks/:taskId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Delete a task that is no longer needed.

Example response

```
{ "success": true, "message": "Task deleted." }
```

Rules

- Hard delete is acceptable for tasks because general task history/audit log is out of scope.
- Allowed only while lifecycle is ACTIVE.

6.10 Galleries and media uploads

List galleries

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/galleries
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	List event galleries and file counts.

Example response

```
{ "success": true, "data": [{ "id": "GALLERY_ID", "eventId": "EVENT_ID", "photoCount": 120, "videoCount": 8 }] }
```

Get gallery

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/galleries/:galleryId
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Get gallery details and paginated media metadata.

Example response

```
{ "success": true, "data": { "id": "GALLERY_ID", "eventId": "EVENT_ID", "files": [] }, "pagination": { "page": 1, "limit": 50, "total": 128, "totalPages": 3 } }
```

Rules

- Do not return upload/view token hashes.

Rotate gallery links

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/galleries/:galleryId/links/rotate
Authentication	Access token + active membership
Allowed actor	Admin only
Success status	200 OK
Purpose	Invalidate previous view/upload tokens and issue fresh links.

Example response

```
{ "success": true, "data": { "viewUrl": "https://.../gallery/view/<new-token>", "uploadUrl": "https://.../gallery/upload/<new-token>" } }
```

Rules

- Return raw tokens only once; store only hashes.
- Rotation invalidates previous links immediately.
- Allowed only while lifecycle is ACTIVE or COMPLETED; archived uploads stay disabled.

Create authenticated upload signature

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/galleries/:galleryId/uploads/signature
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Issue a short-lived signed Cloudinary upload authorization.

Request

```
{ "mediaType": "PHOTO", "fileName": "mehendi.jpg", "contentType": "image/jpeg", "bytes": 2500000 }
```

Example response

```
{ "success": true, "data": { "cloudName": "...", "apiKey": "...", "timestamp": 1794220000, "signature": "...", "folder": "weddingos/...", "uploadParams": {} } }
```

Rules

- Allowed in ACTIVE and COMPLETED; blocked in ARCHIVED.
- Enforce configured MIME type, byte-size, count and rate limits.
- Never expose Cloudinary API secret.

Complete authenticated upload

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/galleries/:galleryId/uploads/complete
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	201 Created
Purpose	Verify the uploaded Cloudinary asset and persist a galleryFiles record.

Request

```
{ "publicId": "...", "assetId": "...", "mediaType": "PHOTO" }
```

Example response

```
{ "success": true, "data": { "fileId": "FILE_ID", "status": "ACTIVE", "secureUrl": "https://..." } }
```

Rules

- Backend verifies the asset against Cloudinary using server credentials; never trust client-supplied secureUrl/bytes/dimensions alone.
- Allowed in ACTIVE and COMPLETED; blocked in ARCHIVED.

Delete one gallery file

Property	Specification
Method / path	DELETE /api/v1/weddings/:weddingId/gallery-files/:fileId
Authentication	Access token + active membership
Allowed actor	Admin only
Success status	200 OK

Property	Specification
Purpose	Soft-delete one gallery file and remove/disable its Cloudinary asset.

Example response

```
{ "success": true, "message": "Gallery file deleted." }
```

Rules

- Managers and guests cannot delete gallery files.
- Keep a tombstone record for operational traceability.

Bulk-delete gallery files

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/gallery-files/bulk-delete
Authentication	Access token + active membership
Allowed actor	Admin only
Success status	200 OK
Purpose	Delete a selected set of files.

Request

```
{ "fileIds": ["FILE_ID_1", "FILE_ID_2"] }
```

Example response

```
{ "success": true, "data": { "deletedCount": 2, "failedCount": 0 } }
```

Rules

- Limit batch size (recommended maximum 100).
- Validate every file belongs to the wedding before performing any deletion.

View public gallery

Property	Specification
Method / path	GET /api/v1/public/galleries/:viewToken
Authentication	Public view token
Allowed actor	Anyone with view link
Success status	200 OK
Purpose	List gallery files and download metadata.

Example response

```
{ "success": true, "data": { "eventName": "Wedding", "files": [{ "id": "FILE_ID", "mediaType": "PHOTO", "secureUrl": "https://...", "format": "jpg" }] },
"pagination": { "page": 1, "limit": 50, "total": 80, "totalPages": 2 } }
```

Rules

- View link permits viewing and individual downloads only.
- Links are bearer credentials and may be forwarded.
- Gallery remains viewable when wedding is COMPLETED or ARCHIVED.

Create guest upload signature

Property	Specification
Method / path	POST /api/v1/public/galleries/:uploadToken/uploads/signature

Property	Specification
Authentication	Public upload token
Allowed actor	Anyone with upload link
Success status	200 OK
Purpose	Issue short-lived signed Cloudinary authorization using an event-specific upload link.

Request

```
{ "mediaType": "PHOTO", "fileName": "photo.jpg", "contentType": "image/jpeg", "bytes": 1800000 }
```

Example response

```
{ "success": true, "data": { "cloudName": "...", "apiKey": "...", "timestamp": 1794220000, "signature": "...", "uploadParams": {} } }
```

Rules

- Allowed in ACTIVE and COMPLETED; blocked in ARCHIVED.
- Apply per-IP/token rate limits and configured file constraints.
- Upload link is a bearer credential and may be forwarded.

Complete guest upload

Property	Specification
Method / path	POST /api/v1/public/galleries/:uploadToken/uploads/complete
Authentication	Public upload token
Allowed actor	Anyone with upload link
Success status	201 Created
Purpose	Verify guest-uploaded Cloudinary asset and create galleryFiles record.

Request

```
{ "publicId": "...", "assetId": "...", "mediaType": "PHOTO" }
```

Example response

```
{ "success": true, "data": { "fileId": "FILE_ID", "status": "ACTIVE" } }
```

Rules

- Verify the upload token and Cloudinary asset server-side.
- Do not store uploader name; PRD specifies anonymous guest uploads.
- Allowed in ACTIVE and COMPLETED only.

6.11 Dashboard and public wedding page

Get dashboard

Property	Specification
Method / path	GET /api/v1/weddings/:weddingId/dashboard
Authentication	Access token + active membership
Allowed actor	Admin or Manager
Success status	200 OK
Purpose	Return upcoming events, pending/in-progress tasks, financial summary and RSVP estimates.

Example response


```
{
  "success": true,
  "data": {
    "upcomingEvents": [],
    "tasks": { "pending": 8, "inProgress": 3 },
    "expenses": { "totalAgreedPaise": 8000000, "paidPaise": 2000000, "outstandingPaise": 6000000 },
    "rsvps": { "invitedFamilies": 100, "yesFamilies": 60, "potentialAttendance": 300, "yesAttendanceEstimate": 180 }
  }
}
```


Rules

- Use active expense records and non-voided payments.
- Potential attendance = invited families x 3. RSVP attendance estimate = YES families x 3.
- Do not persist derived totals as authoritative financial records.

Get public wedding page

Property	Specification
Method / path	GET /api/v1/public/weddings/:token
Authentication	Public unlisted page token
Allowed actor	Anyone with page URL
Success status	200 OK
Purpose	Return safe wedding overview, fixed welcome message and public event list.

Example response


```
{
  "success": true,
  "data": {
    "brideName": "Bride",
    "groomName": "Groom",
    "weddingDate": "2027-02-15T00:00:00Z",
    "venue": { "name": "Grand Hall", "address": "Kanpur" },
    "welcomeMessage": "We are delighted to celebrate with you.",
    "events": [{ "name": "Wedding", "startAt": "2027-02-15T18:00:00Z", "venue": {} }]
  }
}
```


Rules

- Return only guest-safe details; never return internal notes, guest lists, RSVP counts, expense/vendor data or manager assignments.
- Event list includes scheduled events and only guestInfo fields.
- The unlisted URL is not private authentication; anyone holding the link can view its payload.

Rotate public page link

Property	Specification
Method / path	POST /api/v1/weddings/:weddingId/public-page-link/rotate
Authentication	Access token + active membership
Allowed actor	Admin only
Success status	200 OK
Purpose	Rotate the unlisted landing-page token and invalidate the prior link.

Example response

```
{ "success": true, "data": { "publicPageUrl": "https://.../wedding/<new-token>" } }
```

Rules

- Return raw token only once and store only its hash.
- Allowed while lifecycle is ACTIVE.

7. Standard response and error catalogue

7.1 HTTP status usage

HTTP status	Meaning	Typical use
200	Success	Read, update, accepted action, refresh.
201	Created	New account, wedding, event, invitation, guest family, expense, task, payment, gallery file.
204	No content	Reserved; v1.0 generally uses the standard JSON envelope instead.
400	Malformed request	Invalid JSON, malformed path parameter, invalid pagination parameter.
401	Unauthenticated	Missing, invalid or expired access token; invalid refresh cookie.
403	Forbidden	Authenticated but role/membership insufficient; email not verified.
404	Not found	Resource missing, outside wedding scope, deleted token, or intentionally hidden resource.
409	Conflict	Duplicate membership, admin limit, wedding already created, open tasks block manager removal, invalid lifecycle transition.
422	Validation/business rule	Invalid field values, immutable amount update, payment exceeds outstanding, RSVP for non-invited event.
429	Rate limited	Auth, public token, upload authorization or email abuse protection.
500	Server error	Unexpected failure; include requestId but no internal stack trace.

7.2 Stable application error codes

Code	Meaning
VALIDATION_ERROR	One or more request fields failed validation.
INVALID_CREDENTIALS	Email/password pair is invalid.
EMAIL_NOT_VERIFIED	Email/password account must be verified before login.
TOKEN_INVALID_OR_EXPIRED	A one-time or bearer token is invalid, expired or already used.
SESSION_REVOKED	Refresh session is revoked or no longer current.
WEDDING_ALREADY_CREATED	User already created/owns a wedding.

Code	Meaning
WEDDING_ACCESS_DENIED	No active membership for the requested wedding.
ROLE_FORBIDDEN	Role cannot perform this operation.
WEDDING_NOT_ACTIVE	Operation is blocked by wedding lifecycle.
SECOND_ADMIN_REQUIRED	Lifecycle operation requires two active Admins.
ADMIN_LIMIT_REACHED	Wedding already has two active Admins.
INVITATION_ALREADY_PENDING	A matching pending invitation exists.
ALREADY_A_MEMBER	User already has a membership in this wedding.
MANAGER_HAS_OPEN_TASKS	Manager must have pending/in-progress tasks reassigned before removal.
DUPLICATE_CATEGORY	Category name already exists for this wedding and type.
DUPLICATE_RSVP	RSVP uniqueness conflict; normally resolved by upsert.
IMMUTABLE_FIELD	Request attempted to change a protected field such as agreed amount.
PAYMENT_EXCEEDS_OUTSTANDING	Payment is greater than current outstanding amount.
RESOURCE_WEDDING_MISMATCH	Referenced record does not belong to the active wedding.
UPLOAD_NOT_ALLOWED	Upload is blocked by lifecycle, token or configured media policy.
IDEMPOTENCY_KEY_REUSED	Idempotency key was reused with a different payload.

8. Cross-module workflows and transactions

8.1 Registration and email verification

1. POST /auth/register creates an unverified user and a one-time verification token hash.
2. The email worker sends the verification link; raw token exists only in the outbound message/link.
3. POST /auth/verify-email consumes the token, sets emailVerifiedAt, creates authSessions, returns access token and sets refresh cookie.

8.2 Create wedding atomically

1. Validate verified account and one-wedding ownership rule.
2. Start MongoDB transaction.
3. Insert weddings record with lifecycle ACTIVE and hashed public page token.
4. Insert weddingMembers record with role ADMIN and status ACTIVE for creator.
5. Commit transaction and return workspace; rollback on any failure.

8.3 Invite and accept second Admin / Manager

1. Admin or permitted Manager creates an invitation; service validates role rules and active Admin count.
2. Generate random token, persist token hash and seven-day expiry; create email outbox job.
3. Return shareable URL once to inviter and send email.
4. Invitee registers/logs in and verifies email; invitation preview is public but acceptance is authenticated.
5. POST /invitations/accept transactionally creates membership and marks invitation accepted.

8.4 Event update and guest email fan-out

1. PATCH event validates active wedding membership and event ownership.
2. If guestInfo changes or event becomes CANCELLED, identify active guest families with an INVITED relationship for that event.
3. Insert one emailOutbox job per recipient with a unique idempotency key.
4. Worker sends email and updates delivery status/retry metadata.
5. Internal-only changes do not send guest notification.

8.5 RSVP upsert

1. Resolve guest family by invitation token hash and require active family.
2. Verify event belongs to the same wedding and an active invitation exists.
3. Require wedding lifecycle ACTIVE.
4. Upsert one record on {weddingId, guestFamilyId, eventId}; update response and respondedAt.

8.6 Expense and payment integrity

1. Create expense with positive integer amountPaise; agreed amount becomes immutable.
2. Create payment with Idempotency-Key; calculate outstanding from active non-voided payments.
3. Reject amount greater than outstanding; insert payment and return new totals.
4. For a mistake, Admin voids the payment with a reason; do not overwrite or erase the original record.

8.7 Two-Admin lifecycle transition

1. An active Admin starts a supported transition; their approval counts as the first approval.
2. The other active Admin approves; the initiator cannot approve twice.
3. Use transaction to verify both approvals and update lifecycle.
4. On completion, mark PENDING tasks system-completed.
5. On archive, disable all gallery upload authorization; on restore, return to COMPLETED.

8.8 Direct-to-Cloudinary upload

1. Client requests upload signature using either authenticated membership route or guest upload token route.
2. Backend validates wedding lifecycle, gallery, token/role, media type, configured size limits and rate limits.
3. Backend signs a short-lived upload authorization; browser uploads directly to Cloudinary.
4. Client calls upload complete; backend verifies asset by server-side Cloudinary lookup and stores galleryFiles metadata.
5. The client cannot create trusted metadata by merely sending a URL or asset ID.

9. Security, validation and operational controls

9.1 Input and schema validation

- Use Zod schemas for every body, query and path parameter. Reject unknown fields on security-sensitive and financial endpoints.
- Normalize emails to lowercase and trim whitespace. Validate email format and enforce length bounds.
- Validate ObjectId syntax before database access; validate every reference belongs to the same wedding.
- Use allowlisted sort/filter fields. Do not pass arbitrary client objects directly into MongoDB filters or update operators.
- Validate all dates and require explicit timezone information for date-time input.
- Validate currency amounts as safe positive integers in paise; reject decimals, NaN, infinity and negative values.

9.2 Rate limiting

Endpoint group	Suggested control
Register/login/Google	Per-IP and per-account throttles; progressive delay on repeated failures.
Forgot/reset/resend verification	Per-IP and normalized-email throttles; generic responses where enumeration risk exists.
Invitation preview/accept	Per-IP and token rate limit; do not log raw tokens.
Guest invitation/RSVP	Per-token and per-IP limit; protect family link from brute force.
Gallery signature/complete	Per-token, per-IP, per-wedding and daily quota controls; enforce media constraints.
Vendor directory	Provider quota, response caching and request throttling.

9.3 Token and cookie safety

- Use cryptographically secure random token generation for email verification, password reset, wedding invitations, guest family links, public page links, and gallery links.
- Hash opaque tokens before database storage; compare securely. Do not write raw tokens into logs, analytics, error messages or database documents.
- Refresh cookie: Secure, HttpOnly, Path scoped to auth refresh/logout endpoints; SameSite and CSRF behavior must match actual deployment topology.
- Use strict CORS allowlists for the production frontend origins. Never combine credentials with wildcard Access-Control-Allow-Origin.
- Validate Origin/Referer and use a CSRF defense for cookie-authenticated refresh/logout requests when cross-site cookies are used.

9.4 Data isolation and privacy

- Scope every wedding query by weddingId and membership. For guests, scope through token -> family -> active event invitation -> event.
- Use explicit response serializers for public/guest payloads. Never return internalInfo, manager assignments, guest lists, expenses or vendor notes to guests.
- Avoid leaking whether a family token or invitation token exists; use generic invalid-link responses.
- Sanitize logs; do not log passwords, raw bearer tokens, cookies, reset links, guest invitation links or full payment receipt URLs if they are private.

9.5 Idempotency and concurrency

- Require Idempotency-Key for payment creation and use a persisted key/payload fingerprint to prevent double charges in record-keeping flows.
- Use unique compound indexes as final protection for membership, categories, event invitations and RSVP uniqueness.
- Use MongoDB transactions for multi-document invariants: create wedding + first Admin; accept invitation + membership; create event + gallery; lifecycle transition + system task updates.
- For concurrent payment requests, calculate outstanding and insert inside a transaction or use a conditional atomic guard so the total cannot exceed the agreed amount.
- Use idempotent emailOutbox jobs and BullMQ retry-safe processing. Email delivery is at-least-once; idempotency keys prevent duplicate logical notifications.

9.6 Audit/history boundaries

WeddingOS does not include a general activity log in the MVP. Preserve only the specific history required by the PRD: expense deletion metadata, payment void metadata, vendor soft-removal, membership status, invitation status, and gallery file deletion metadata. Avoid creating a broad audit subsystem unless product scope changes.

10. Database alignment and required model updates

The API design builds on the approved Database Design v1.0. A few implementation-level adjustments are required before schema coding so the API contracts have storage support:

Model area	Required adjustment	Reason
Invitations	Replace managerInvitations with unified weddingInvitations; fields include invitedRole, emailNormalized, tokenHash, invitedBy, expiresAt, status, acceptedBy and acceptedAt.	Same workflow supports ADMIN and MANAGER; Admin capacity can be validated consistently.
One-time tokens	Add oneTimeTokens collection or equivalent secure token records with purpose, userId, tokenHash, expiresAt, consumedAt.	Supports email verification and 30-minute password reset without placing multiple transient secrets on users.
authSessions	Include token/session identifiers, current refreshTokenHash, expiresAt, revokedAt, userAgent/device label, lastUsedAt and rotation metadata.	Supports independent device sessions, rotation and reuse detection.
weddings	Add/confirm unique createdBy, publicPageTokenHash, lifecycle request state and timezone.	Enforces one created wedding per user and supports public page + two-Admin lifecycle.
guestFamilies	Allow invitationTokenHash to be unset on deletion/rotation; keep status and deletion metadata.	Immediately invalidates guest access while preserving historical records.
galleries	Store separate viewTokenHash and uploadTokenHash; support token rotation timestamps.	Separates viewing and uploading capabilities.
payments	Add status, voidedAt, voidedBy and voidReason; add idempotency key or a separate idempotency record.	Preserves correction history and prevents duplicate payment entries.
tasks	Add completionSource and completedAt for system completion during wedding completion.	Distinguishes human task completion from lifecycle automation.
emailOutbox	Add unique idempotencyKey, recipient, event/family references, payload, status, attempts and nextAttemptAt.	Supports reliable event-change emails and retry processing.
guest import preview	Use Redis short-lived preview payload (30 minutes); no persistent collection required for MVP.	Allows preview and explicit commit without partially creating families.

10.1 Important index constraints

- users: unique emailNormalized; unique sparse googleId.
- weddings: unique createdBy; unique sparse publicPageTokenHash.
- weddingMembers: unique {userId, weddingId}; index {weddingId, status, role}.
- weddingInvitations: unique tokenHash; index {weddingId, emailNormalized, invitedRole, status}.
- oneTimeTokens: unique tokenHash; TTL on expiresAt, with application-side consumedAt check.
- categories: unique {weddingId, type, normalizedName}.
- guestEventInvitations: unique {weddingId, guestFamilyId, eventId}.
- rsvps: unique {weddingId, guestFamilyId, eventId}.
- galleries: unique {weddingId, eventId}; gallery link token hashes unique.

- payments: unique idempotency key scoped by wedding/user or endpoint; indexes by {weddingId, expenseId, paidAt}.
- emailOutbox: unique idempotencyKey; index {status, nextAttemptAt}.

TTL indexes are cleanup conveniences, not authorization rules. Always check expiresAt and consumed/revoked state in application code.

11. Implementation sequence

Phase	Build order	Exit criteria
1. Foundations	Express app, /api/v1 router, response/error middleware, Zod validation, request IDs, CORS, rate limits, Mongo connection.	Health route, standardized response/error handling, test harness and OpenAPI skeleton.
2. Identity	Users, oneTimeTokens, authSessions, register/login/Google/verify/refresh/logout/reset /sessions.	Authentication tests, refresh rotation/reuse tests, multi-session tests.
3. Wedding access	Weddings, weddingMembers, weddingInvitations, create/list/detail/update, invitation accept, manager removal.	Ownership uniqueness, Admin cap, role checks, immediate membership revocation tests.
4. Events and guests	Events, categories, guestFamilies, imports, event invitations, public guest page, RSVPs.	Event isolation, guest token scopes, RSVP uniqueness and notification outbox tests.
5. Finance and vendors	Expenses, payments, vendor CRUD/directory adapter.	Immutable agreed amount, payment idempotency, voiding, totals and soft-delete tests.
6. Tasks and lifecycle	Tasks, task reassignment guard, two-Admin lifecycle requests.	Completion/archive/reopen/restore state-machine tests.
7. Galleries and dashboard	Gallery links, Cloudinary signing/completion, file metadata, dashboard aggregates.	Upload lifecycle restrictions, role checks, asset verification and aggregate tests.
8. Hardening	OpenAPI review, integration tests, Playwright flows, security checks, performance and deployment validation.	No cross-wedding access, guest data leakage, token replay or duplicate financial records.

11.1 Testing requirements

- Unit tests for service rules and Zod schemas.
- Integration tests for every endpoint's authentication, role, wedding boundary and lifecycle policy.
- Concurrency tests for RSVP upserts, invitation acceptance, wedding creation ownership and payment outstanding guards.
- Negative tests proving a user cannot change the weddingId in a request to access another wedding.
- Guest tests proving one family cannot view another family or events for which it was not invited.
- Gallery tests for token rotation, archived upload denial, Cloudinary asset verification and Manager delete denial.
- OpenAPI/Swagger contract tests to keep implementation and documentation aligned.

12. Open assumptions and deferred details

The following decisions are made for v1.0 where the product specification did not fully define implementation behavior. They are sensible MVP defaults, not claims that the external provider configuration is already deployed.

Topic	v1.0 decision / assumption	Revisit when
Wedding detail editing	Admins and Managers can edit names, date and main venue while ACTIVE.	Usability feedback or permission review.
Lifecycle restore	ARCHIVED -> COMPLETED; a separate two-Admin REOPEN request is needed to return to ACTIVE.	If product owners intend archive restore to return directly to ACTIVE.

Topic	v1.0 decision / assumption	Revisit when
Lifecycle requests	Initiator counts as first approval; second distinct active Admin approves. Initiator can cancel a pending request.	If approval needs formal expiry or notifications.
Admin availability	Lifecycle transitions require exactly two active Admin memberships; a wedding with only its creator Admin cannot complete/archive.	If single-Admin exception is desired.
Guest RSVP after completion	RSVP writes stop when lifecycle is COMPLETED or ARCHIVED; guest links remain viewable.	If late RSVP updates should continue after completion.
Guest import	Frontend parses CSV, sends JSON rows for preview; Redis stores preview for 30 minutes.	If server-side XLSX support is added.
Invitation re-issue	No manual revoke endpoint. A resend/re-issue process should rotate the token and invalidate prior token; not in v1.0 route catalogue.	When users need resend UX after losing a link.
Gallery limits	File MIME allowlist, per-file max size, video duration and per-token rate limits are configurable; exact numbers remain deployment settings.	Before enabling public uploads in production.
Vendor directory	Provider adapter is defined; exact provider, terms, quota and cache behavior remain undecided.	Before implementing public listing search.
Cookie topology	Prefer same-site routing; otherwise SameSite=None + Secure + strict CORS/CSRF.	When production hostnames and routing are finalized.
Access-token invalidation	Access JWTs remain valid for at most 15 minutes; member removal blocks wedding routes immediately via membership check.	If global immediate access-token revocation is required.
Email delivery	Use emailOutbox + BullMQ/Redis worker; Resend/Nodemailer adapter and templates follow deployment quotas.	Before production notification rollout.

Appendix A. API implementation checklist

- ☐ All routes mounted under /api/v1 and documented in OpenAPI.
- ☐ Auth middleware validates access token; membership middleware validates active wedding membership.
- ☐ Role middleware enforces Admin/Manager actions and guest token middleware enforces event-scoped access.
- ☐ Lifecycle middleware blocks operational writes in COMPLETED and ARCHIVED; gallery uploads have their documented exception.
- ☐ Zod schemas validate body, query and path parameters; reject client-provided system fields.
- ☐ Every wedding-scoped query includes weddingId; every reference is checked against that same wedding.
- ☐ Tokens are random, hashed at rest, single-use where applicable, and never logged.
- ☐ Critical multi-document workflows use MongoDB transactions on a supported replica-set deployment.
- ☐ Payment creation is idempotent and amount-safe; expense agreed amount is immutable.
- ☐ Public and guest serializers expose only allowed fields.
- ☐ Event guest-facing changes create idempotent emailOutbox records.
- ☐ Rate limits, CORS, CSRF, Pino/Winston logging and generic error responses are enabled.
- ☐ Unit, integration and end-to-end tests cover both allowed and denied access paths.

Appendix B. Final API design decisions

Decision	v1.0 baseline
Wedding routes	Nested /weddings/:weddingId for authenticated wedding operations.
Registration vs creation	Separate account registration and wedding creation.

Decision	v1.0 baseline
Email verification	Required; successful verification auto-logs in.
Google	First login creates account; verified matching email links identities.
Admin invitations	Dedicated role via unified weddingInvitations; explicit acceptance.
Invitation expiry	Seven days; single-use; hash stored.
Access/refresh	15-minute access JWT; seven-day sliding refresh JWT with rotation.
Sessions	Multiple devices; list/revoke one/revoke all.
Password reset	Generic forgot-password response; 30-minute link; reset revokes all sessions.
Wedding ownership	One created wedding per user; user may join other weddings.
Roles	Two equal Admins maximum; Managers have operational access.
Lifecycle	Two distinct Admin approvals; ARCHIVED restores to COMPLETED.
Money	Integer paise; agreed amount immutable; payments append-only with Admin void.
Guests	Family-level records; guest bearer link; one RSVP per family/event.
Galleries	Separate view/upload bearer links; Cloudinary signed direct uploads.
Notifications	Guest event-change emails via outbox + background worker.